VirSME

Application Performance Audit

Plain-English Guide for Management & Non-Technical Readers

Overall performance maturity

7 / 10

for a GCC-wide, multi-tenant SaaS target

Audit date: 17 August 2026

> **Important interpretation**
>
> This score does not mean VirSME is only 70% functional. The audit says the product is functionally broad and contains several strong engineering fixes. The score reflects how ready the current architecture is for reliable, measurable, multi-tenant growth across the GCC.

Prepared from the technical VirSME Application Performance Audit. This guide simplifies the audit without replacing its detailed engineering evidence.

START HERE

How to Read This Guide

The original audit is written for engineers, SRE, database, cloud and security teams. This version translates the same findings into business language so a normal user can understand what is happening, why it matters, and what should be done next.

Audit label	Plain-English meaning
MEASURED	A number was produced directly during the audit.
CODE-VERIFIED	The behaviour was read directly from source code or configuration.
INDICATED	The code strongly suggests the issue, but runtime proof is still needed.
MEASUREMENT REQUIRED	The audit could not prove the real production number without telemetry or a seeded staging environment.

> **What the audit did NOT do**
>
> It did not connect to production, query the production database, load-test real users, or inspect secrets or personal data. Therefore, actual API latency, real page-load speed, current error rate, production CPU/memory under load, and true concurrent-user capacity remain to be measured.

The one-sentence conclusion

VirSME can work for today's small customer base, but the current design has several structural limits that make a Saudi/GCC multi-tenant launch risky unless the database, deployment, observability, background jobs, file storage, tenant configuration and frontend delivery are upgraded.

WHAT MANAGEMENT NEEDS TO KNOW

Executive Summary

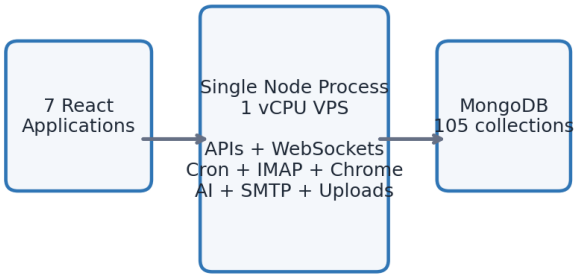
7/10	**Overall GCC multi-tenant performance maturity** The product contains good local optimisations, but performance engineering is not yet a system-wide capability.
1 vCPU	**Current production shape indicated by the audit** One server/process is expected to serve APIs, WebSockets, cron jobs, email, PDFs, AI calls and uploads. That creates a single point of failure.
51 / 86	**Tenant-scoped models without an owner-leading index** As more tenants are added, many queries risk scanning platform-wide data instead of staying limited to one tenant.
0	**Tests / CI / observability stack** The audit found no automated tests, CI workflow, APM/tracing/metrics, or structured logging. Problems can be hard to detect and changes are harder to prove safe.
~5 hrs	**Estimated engineering time for 20 quick wins** The source audit identifies a group of low-effort changes that can remove several high-impact payload, security and recurring-query problems.

What is already good

- WebSocket events are usually targeted to rooms rather than broadcast to everyone.
- Message models have several useful compound indexes aligned with real communication queries.
- The WhatsApp webhook acknowledges quickly and uses an idempotency mechanism to block duplicates.
- A shared Puppeteer browser pool, pooled SMTP transport and slimmed message list payloads show that real production incidents have been addressed thoughtfully.
- Most models already contain an owner field, so the core idea of tenant ownership exists in the data model.
- Each app has an initial loader that improves perceived performance while large JavaScript bundles are loading.

ARCHITECTURE IN SIMPLE TERMS

How VirSME Runs Today



Main structural concern: too many critical workloads depend on one process and one server.

This works at small scale, but it limits redundancy, safe scaling and fault isolation.

The audit describes a shared database and a single Node process handling many different responsibilities. This is efficient for an early-stage product, but it makes failures more contagious: a heavy PDF render, email process, AI call or scheduled job can compete with normal user requests on the same server.

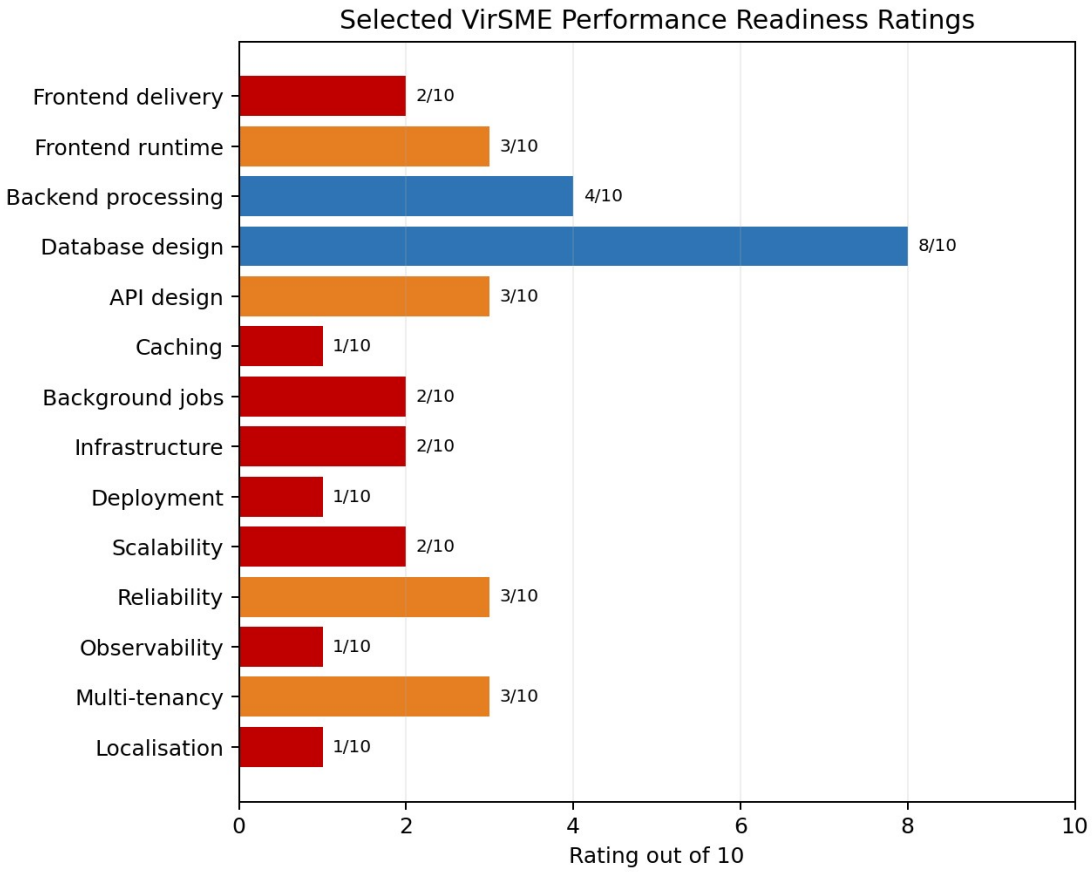
> ### Why this matters
>
> Imagine one person answering the reception phone, preparing payroll, printing all documents, checking every mailbox and watching every live chat at the same time. It may work with a small workload, but adding more customers does not simply make that person faster. The architecture needs separate workers, shared state and more than one web replica.

Current platform facts highlighted by the audit

Item	Audit finding	What it means
Backend	Node.js / Express, single process	All major workloads share one application process.
Database	MongoDB, 105 collections	Large shared data model; indexing strategy becomes crucial as tenants grow.
Real-time	Socket.IO without shared adapter	A second API replica cannot safely share room/presence state yet.
Files	698 MB on local server disk	Files are tied to one server and not naturally available to additional replicas.
Scheduling	Multiple in-process cron jobs	Running more replicas would duplicate scheduled work unless leadership/queueing is added.
Frontend	7 React SPAs	Several applications ship very large first-load bundles.

WHERE THE PLATFORM IS WEAKEST

Performance Scorecard



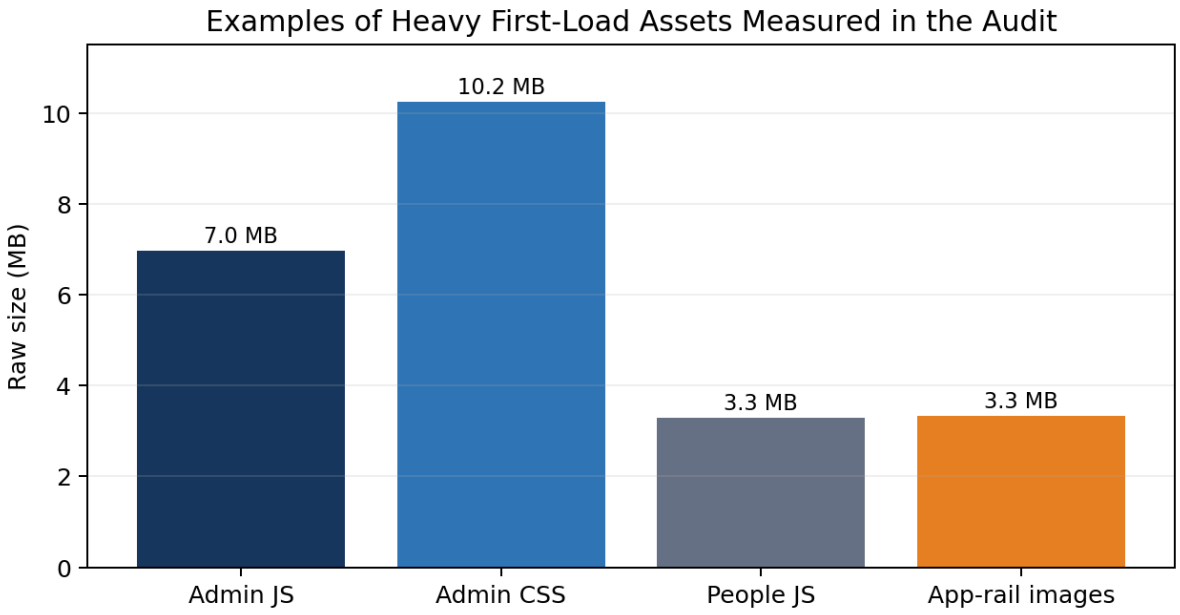
Ratings are from 1 (critical deficiency) to 10 (production-excellent for a GCC multi-tenant SaaS). Database design is now rated 8/10; several other foundational areas remain rated 1-4/10 and should still be prioritised.

Area	Rating	Plain-English concern
Frontend delivery	2/10	Users download too much JavaScript/CSS before opening one screen.
Database design	8/10	Strong overall database design, with targeted tenant-index improvements still recommended before larger-scale multi-tenant growth.
Caching	1/10	No shared cache and almost no client-side query caching.
Infrastructure	2/10	Single-server shape with no redundancy, CDN or horizontal scale story.
Deployment	1/10	No CI/CD, container, rollback process or versioned deployment artefact.
Observability	1/10	No structured metrics/traces/APM to prove performance or detect issues early.
Localisation readiness	1/10	Timezone/currency are hardcoded for Pakistan and there is no i18n/RTL foundation.

SYMPTOMS, NOT CODE

What a Normal User May Experience

The audit cannot prove current production latency without telemetry, but it identifies multiple code paths that can create slowness or instability as data and concurrent usage increase.



User-facing symptom	Likely technical reason identified by the audit
Slow first page load	Very large JavaScript/CSS bundles, oversized 28px icons, unnecessary font/script requests and no route-level splitting.
Dashboard slows as history grows	Attendance queries can full-scan the collection because the available index does not match owner + date queries.
Repeated loading/spinners	No data caching/deduplication layer and many hand-built API calls.
Background traffic even when idle	Six 30-second polling loops plus live Socket.IO; one poll uses a seven-query endpoint for one count.
Chat typing/scrolling becomes heavy	Very large components, many state variables and no list virtualisation.
Login can occasionally stall	Many sequential operations plus a third-party geo-IP call with up to a 4-second timeout.
A heavy task affects unrelated screens	PDF generation, AI, email, IMAP and API traffic share the same process/server.

> **Important limitation**
>
> The audit marks actual page-load times, API p50/p95/p99, error rates and concurrent-user capacity as measurement required. These symptoms are architectural risks, not claimed production timings.

PRIORITY FINDINGS

The 10 Most Important Problems - in Plain English

1. Tenant indexing is incomplete

51 of 86 owner-scoped models lack an owner-leading index. At many tenants, a request for one company can become work proportional to all companies' data. Severity: Critical.

2. Attendance dashboard queries can full-scan

The main attendance index starts with employee, while dashboard queries filter owner and date. The index prefix therefore cannot serve the hot query. Severity: Critical.

3. Every authenticated request can hit the database repeatedly

Authentication performs 1-3 uncached database reads per request; a workspace load fires many requests. Severity: Critical.

4. Employee list payload is too broad

The employee list is unpaginated and returns many fields, including sensitive/reset-token-related fields that should not be in a general list. Severity: Critical.

5. No response compression or rate limiting

Large JSON transfers stay large and important unauthenticated/auth endpoints lack a protective traffic limit. Severity: Critical.

6. File uploads expose reliability/security risk

Several upload routes are unauthenticated or lack file-size limits; one buffers uploads into memory. Severity: Critical.

7. One server/process is doing too much

APIs, WebSockets, cron jobs, IMAP, Chrome/PDF work, AI and email all share one 1-vCPU process. Severity: Critical.

8. Frontend bundles are extremely heavy

Admin measured ~6.96 MB raw JS + ~10.25 MB raw CSS, and several apps ship multi-megabyte entry bundles. Severity: Critical.

9. No tests, CI or observability safety net

There is no reliable automated way to prove that a change works, catch regressions, or diagnose production performance. Severity: High/Critical foundation.

10. Saudi/GCC configuration is not ready

Asia/Karachi is hardcoded in 24 places and PKR in 11; there is no internationalisation/RTL foundation. Severity: Launch blocker.

WHY “SECURITY” AND “SPEED” OVERLAP

Reliability & Security Risks That Also Affect Performance

Several findings are not just technical cleanliness issues. They can directly create downtime, data exposure, or expensive recovery work.

Risk	Why management should care	Recommended direction
Single point of failure	One process/server failure can affect all products at once.	Move background jobs out, use shared state, then add multiple API replicas.
Open/unbounded uploads	An attacker or accidental large upload can consume RAM/disk and take the service down.	Authenticate routes, require single-use onboarding token where needed, enforce size/file limits.
Global socket broadcasts	Eight broadcasts can reach every connected tenant and may expose cross-tenant event information.	Use tenant-scoped rooms and strict socket origin controls.
Reset tokens in list payload	General employee response can expose sensitive reset state to users who should not receive it.	Use positive field allow-lists and separate sensitive endpoints.
No rate limiting	Login, OTP, reset, upload, search and AI-triggering endpoints can be abused cheaply.	Introduce endpoint-specific rate limits.
No rollback/test pipeline	A bad release may require manual recovery and has no automatic regression net.	CI/CD, tests, versioned artefacts and rollback procedure.

Management view

Performance work should not be separated from reliability and security here. Many of the highest-value changes improve all three at the same time.

WHAT BLOCKS REGIONAL GROWTH

Saudi / GCC Launch Readiness

The audit explicitly evaluates VirSME against a Saudi-launched, GCC-wide, multi-tenant SaaS goal. The main blockers are structural rather than cosmetic.

Launch area	Current issue	What “ready” looks like
Tenant isolation & scale	Owner fields exist, but many owner-leading indexes/limits do not.	Tenant-scoped indexes, quotas, per-tenant metrics and no noisy-neighbour impact.
Timezone & currency	Asia/Karachi hardcoded 24 times; PKR hardcoded 11 times.	Timezone/currency stored per tenant and used by all schedulers and calculations.
Arabic / RTL	No i18n or RTL framework found.	Shared UI foundation, translation layer, RTL support and regional date/calendar rules where required.
Regional performance	No CDN and origin is indicated as non-GCC.	Regional object storage/CDN and API/database in a suitable GCC/Saudi region.
Availability	Single VPS cannot provide robust redundancy.	At least two API replicas, separate workers, health/readiness checks and load balancer.
Data & files	Uploads live on local server disk.	Object storage with backup, lifecycle, CDN and replica-independent access.

A specific timing problem

Because daily scheduling is hardcoded to Asia/Karachi, the audit warns that Saudi end-of-day jobs could run around 21:00 local time - before the Saudi working day is actually finished.

LOW EFFORT, HIGH CONFIDENCE

Quick Wins - What Can Improve Fast

The technical audit lists 20 quick wins with an estimated total engineering effort of roughly five hours. The purpose is not to “finish the platform” in five hours; it is to remove obvious waste and close several high-impact exposures before larger architecture work.

Quick action	Business effect	Audit effort
Attendance owner/date index	Stops the hottest dashboard path from full-scanning attendance.	~5 min
Resize four app icons	Cuts about 3.3 MB of oversized images per cold app load.	~30 min
Enable response compression	Projected 70-85% reduction in JSON transfer.	~10 min
Fix Google Fonts	Removes unused families and actually loads Inter.	~15 min
Remove gpteng.co production script	Removes an unnecessary third-party module from sensitive apps.	~5 min
Add upload size limits	Closes unbounded upload DoS path.	~15 min
Protect two upload routes	Closes unauthenticated PII/file surfaces.	~15 min
Use positive employee field list	Projected 80-90% smaller list payload and removes reset-token leakage.	~30 min
Remove the heaviest 30-sec approval poll	Stops repeated seven-query work for one badge count.	~20 min
Fix invalid EmployeeSession index	Restores intended active-session constraint.	~5 min
Add lean() to auth lookup	Reduces object-hydration overhead on every authenticated request.	~5 min
Disable automatic production index creation	Removes repeated createIndex calls on boot; manage indexes deliberately.	~5 min

Other quick wins in the audit cover dependency clean-up, cache headers, AI model/token settings, AI timeouts, health/readiness endpoints, browser-data refresh and tenant-scoped socket broadcasts.

WHAT TO DO AND WHEN

Recommended Roadmap

This week

- Complete all 20 quick wins.
- Fully authenticate and bound the open upload routes.
- Add rate limiting to login, OTP, reset, upload and search.
- Start observability first: structured logs with request/tenant IDs, RED metrics and MongoDB slow-query profiling.

First 30 days

- Add the 16 priority tenant/database indexes.
- Paginate and shrink employee lists; batch multiple workspace summary calls.
- Cache authentication context with explicit revocation.
- Replace six frequent pollers with a socket-driven counts channel.
- Reorder salary-slip aggregation for page-first joins.
- Establish CI quality gates: typecheck, lint, build and core API/tenant tests.
- Add real health/readiness probes and move geo-IP work off the login critical path.

Days 31-60

- Route-level frontend code splitting and remove the admin Tailwind safelist.
- Introduce Redis for shared Socket.IO/state and a leader lock for scheduled jobs.
- Introduce a job queue; move salary-slip email and bulk PDF first.
- Complete tracing/RUM/AI telemetry and then harden/resize AI calls.
- Finish tenant-scoped sockets/CORS and remove unused heavy backend dependencies.

Days 61-90

- Move the 698 MB of uploads to object storage + CDN; resize images on upload.
- Replace recursive hierarchy database walks with scalable traversal.
- Move base64 email attachments out of MongoDB documents.
- Make timezone and currency tenant-specific - the audit says to gate GCC launch on this.
- Expand Playwright journey coverage and verify live edge caching/compression.

FROM STARTUP ARCHITECTURE TO GCC SAAS

Three to Six Months - Target Architecture

Workstream	Outcome
Regional deployment	Containerised, load-balanced deployment in a suitable Saudi/GCC region with at least two API replicas and separate workers.
Tenant governance	Per-tenant quotas, query/job limits and noisy-neighbour containment.
Search	Replace broad regex scanning with proper search indexes.
Localisation	Build i18n/RTL foundation and then complete Arabic support.
Quality & capacity	Run repeatable load tests and establish a measured capacity envelope.
Data lifecycle	Define archival and retention for attendance, sessions, messages, notifications and onboarding events.

> **Sequence matters**
>
> Do not move to a more expensive regional infrastructure first and assume the architecture is fixed. The audit intentionally sequences shared state, queues and object storage before multi-replica regional deployment.

Longer-term engineering clean-up

- Extract duplicated mega-components into shared packages before paying for the Arabic/RTL retrofit multiple times.
- Consolidate overlapping UI libraries and icon sets.
- Break large chat/email components into smaller memoised units and virtualise long lists.
- Consider separating communications workloads (WhatsApp/email/chat) from HRMS/payroll if telemetry confirms their scaling profile warrants it.

RECOMMENDED PERFORMANCE TARGETS

What “Good” Should Look Like

These are targets, not current measurements

The audit explicitly says current production values do not yet exist for these metrics because observability is missing. They become meaningful only after telemetry is installed.

Metric	Recommended target	Why a normal user cares
First-route JS	<= 250 kB gzip per app	Less code to download/parse before the screen becomes usable.
Total first load	<= 800 kB cold cache	Faster mobile experience and lower data use.
LCP	<= 2.5 s p75 on 4G in Riyadh	Main content appears quickly for most users.
Workspace requests	<= 8 (from 18)	Less fan-out and less repeated authentication/database work.
Idle background traffic	<= 2 requests/min/tab (from ~13)	Less wasted bandwidth and backend load.
API p95	<= 400 ms in-region	95% of requests should complete within a reasonable backend time.
DB query p95	<= 50 ms	Database should not dominate request time.
Error rate	<= 0.1% 5xx	Normal operation should very rarely produce server errors.
Availability	99.9% monthly after multi-replica architecture	Service remains reachable even when a component fails.
Bulk PDF - 100 employees	<= 5 min without hurting API p95	Heavy back-office work should not freeze normal users.

THE BIGGEST EVIDENCE GAP

What We Still Do Not Know

The audit is unusually careful about evidence. Several important business questions cannot be answered from source code alone and need real telemetry or a seeded staging environment.

- Actual API p50/p95/p99 latency.
- Actual MongoDB query execution plans and slow-query distribution in production.
- Real CPU and memory behaviour under concurrent load.
- Current production error rates and uptime.
- Real browser Core Web Vitals (LCP, INP, CLS) from GCC locations/devices.
- True concurrent-user and WebSocket capacity.
- Background job and bulk-PDF durations under realistic volumes.
- AI latency, token consumption and cost per workflow/tenant.
- Actual nginx/Apache compression and cache headers at the live edge.
- Network latency from Riyadh, Dubai and Doha to the current origin.

How to measure safely

The audit proposes a dedicated staging environment using only synthetic data, with external services stubbed or sandboxed. It then recommends smoke, baseline, normal, peak, stress, 12-hour soak, spike, batch-overlap and failure-injection tests.

> **Release-blocking failures in the proposed load plan**
>
> Examples include any cross-tenant data leakage, OOM/pool-exhaustion 5xx errors, sustained p99 above 3x target, large memory growth during soak tests, silently dropped jobs, or heavy batch work degrading interactive p95 by more than 50%.

QUESTIONS TO ASK THE ENGINEERING TEAM

Management Action Checklist

Decision area	Management question
Immediate safety	Have the unauthenticated/unbounded upload surfaces been closed? Is endpoint rate limiting live?
Evidence	Can we see p50/p95/p99, error rate, CPU, memory, Mongo slow queries and tenant-level dashboards?
Database scale	Have the owner-leading indexes been added and verified with explain() rather than only created?
Frontend	Are first-route bundles below agreed budgets, and is route-level code splitting implemented?
Deployment	Does every pull request run typecheck/lint/test/build? Can production roll back safely?
Jobs	Are PDFs, bulk email, AI and scheduled work running outside the user-facing API process with retries/DLQ?
Shared state	Is Redis/shared state in place so a second API replica behaves correctly?
Files	Are uploads in object storage with CDN/backup/lifecycle rather than local app disk?
GCC launch	Is timezone/currency tenant-specific? Is Arabic/RTL planned? Is regional infrastructure chosen after data-residency review?
Capacity	Has the system passed realistic multi-tenant load, soak and failure-injection tests?

Recommended governance

Treat observability and automated testing as gates for the optimisation programme. Otherwise improvements may be real but remain unproven, and regressions can be introduced without detection.

SIMPLE DEFINITIONS

Glossary for Non-Technical Readers

Term	Meaning
API	The doorway the app uses to request or save data.
Cache	A short-term copy of data used to avoid repeating the same expensive work.
CDN	A network of servers closer to users that delivers files faster.
CI/CD	Automated checks and deployment process that makes releases repeatable and safer.
Database index	Like the index at the back of a book: it helps find the right records without reading everything.
Full collection scan / COLLSCAN	The database reads many or all records to find the requested items.
IXSCAN	The database used an index to find the relevant records more efficiently.
Horizontal scaling	Adding more application servers instead of making one server bigger.
Load balancer	Routes users across multiple healthy application servers.
Multi-tenant	One platform serves many customer organisations while keeping their data/workloads logically separated.
Observability	Logs, metrics and traces that show what the system is doing and why it is slow or failing.
Object storage	Cloud file storage designed for documents/images, independent of any one app server.
p95 latency	95% of requests are faster than this number; only the slowest 5% are worse.
Queue / worker	Heavy jobs wait in a durable line and are processed separately from interactive user requests.
Rate limiting	Restricts how many requests a user/device can make in a time window.
Redis	A fast shared data store often used for caching, sessions, real-time coordination and queue infrastructure.
Route-level code splitting	The browser downloads code for the screen a user opens instead of the entire product at once.
WebSocket / Socket.IO	A live connection used for chat, notifications and real-time updates.

BOTTOM LINE

Final Management Summary

The audit shows a broad product with several thoughtful fixes and a workable foundation for the current scale. The concern is that the current architecture still behaves like a single-customer / single-server system in several important places, while the business goal is a regional multi-tenant SaaS.

The highest-return programme is therefore:

- Measure first: logs, metrics, traces, real-user monitoring and slow-query visibility.
- Fix tenant database indexes and shrink/limit the hottest APIs.
- Apply the quick payload, upload, rate-limit and socket-scope fixes immediately.
- Reduce frontend first-load weight and unnecessary background polling.
- Move shared state and heavy jobs out of the single API process.
- Move files to object storage/CDN and only then add multiple regional API replicas.
- Make timezone/currency tenant-aware and build the localisation/RTL foundation before GCC launch.
- Prove capacity with multi-tenant load, soak and failure-injection tests.

> **Business outcome if executed in sequence**
>
> The platform becomes easier to measure, safer to change, faster for users, harder to overload, capable of adding replicas, and more credible for a Saudi/GCC multi-tenant launch.

Source basis: VirSME Application Performance Audit dated 17 August 2026. For technical implementation details, exact file/line evidence, validation commands and all 35 optimisation recommendations, refer to the original audit.